



**Politecnico
di Torino**

MASTER'S DEGREE IN
CYBERSECURITY

LABORATORY 5
CLOUD FORENSICS

Author:
Stefano Pietro Sternativo
s346910

Professor:
Andrea Atzeni

Academic Year: 2025/2026

Last update: 2026-07-22

Contents

1	Purpose and Scenario	2
1.1	Investigation Objectives (from the Engagement Letter)	2
1.2	What We Already Have, and What We Still Need to Find Out	3
1.3	Evidence & Access Manifest	4
2	Tool Procurement — A Role-Based Exercise	4
2.1	Required Capability Areas	5
2.2	Background: Object Storage, S3, and Why It Matters Here . .	5
2.3	Background: What Docker Is, and Why We Use It Here . . .	6
2.4	Pre-approved Tools	6
3	Setting Up the Forensic Environment	6
3.1	Quick VM Setup via Pre-built .ova Image	7
3.2	Base Forensic VM — Distribution Choice and Write-Blocking (recap from Laboratory 2)	8
3.3	Installing Docker in the Forensic VM	8
3.4	Setting Up a Local S3-Compatible Emulator (MinIO)	9
3.5	Configuring <code>aws cli</code> Against the Local Endpoint	10
4	First Contact — Direct Interaction with the Bucket	11
4.1	Creating the Test Bucket and Uploading Objects	12
4.2	Integrity Verification of the Retrieved Object	12
4.3	Listing Object Metadata	13
5	Analysis of Provided Log Material	13
5.1	Initial Survey	14
5.2	The Missing Data-Plane Events	15
5.3	Cross-Validating the Finding with a Second, Independent Method	15
5.4	Reconstructing the Attacker's Actions	17
6	Redundancy and Primary Copy Identification	17
6.1	Inspecting the Version Export	18
6.2	The Replication Trap	18
6.3	Chain of Custody Documentation	19
7	Cloud Service Model Comparison	19
8	Legal & Trust Module	20
9	Wrap-Up	22

1 Purpose and Scenario

CLASSIFIED — Memorandum

Memorandum — Commandant Isabella Ricci, Polizia Postale

To: Digital Forensics Unit — All Roles

Subject: New Engagement — NovaDynamics S.p.A., Operation SILENT BUCKET

Team; three months after the ransomware incident, NovaDynamics S.p.A. completed a migration of its production workloads to a cloud provider, seeking better resilience. Yesterday, their CISO Dr. Silvia Valenti reported that a batch of confidential client contracts has surfaced for sale on an underground forum. The affected data was stored in a cloud object storage bucket used by NovaDynamics' billing service (IaaS-hosted VM) and a separate customer-facing PaaS application.

NovaDynamics has authorized limited API-level access to their cloud environment, mediated through their cloud service provider, under a new Forensic Analysis Assignment dated 18 May 2026. As in past engagements, physical access to any hardware is out of question.

I expect the team to determine what happened, when, and whether the evidence available to us is sufficient to support a judicial case, given the peculiar constraints of this environment. Coordinate explicitly, and document exactly *what you could and could not verify*.

Commandant Isabella Ricci — Polizia Postale, Digital Forensics Unit

This laboratory scope covers: the IaaS/PaaS/SaaS distinction and its impact on evidence availability; interaction with a cloud object store via API; interpretation of provider-supplied event logs and their inherent gaps; identification of the authoritative (“primary”) copy among redundant/replicated objects; and the legal and trust implications of relying on cloud-provider-mediated evidence.

1.1 Investigation Objectives (from the Engagement Letter)

NovaDynamics S.p.A. formally requests that the team:

- Determine which cloud service model (IaaS/PaaS/SaaS) applies to each affected component, and what evidence is realistically obtainable from each.

- Reconstruct a timeline of the exfiltration event using the log material the cloud provider has made available.
- Identify which of the several copies/versions of the affected storage object is authoritative, and document this in the chain of custody.
- Assess the legal admissibility of cloud-provider-mediated evidence, and discuss the jurisdictional complications raised by the case.
- Deliver a short technical report addressing the above, including an explicit statement of the investigation's limitations.

1.2 What We Already Have, and What We Still Need to Find Out

In a disk or memory investigation, the evidence is a single artefact (a disk image, a memory dump) that you acquire once and then examine offline, at your own pace, with full control over the copy. In a cloud investigation, there is no single artefact to acquire: what NovaDynamics' cloud provider gives you is a set of *exports*, such as logs, metadata listings, billing reports, each produced by a different internal system of the provider, on a different schedule, with different blind spots.

Your job is not just to *read* these exports, but to figure out *what they can and cannot tell you*, and to fill the gaps with reasoning, not with more raw disk bytes (because there are none you can access).

That is exactly why this laboratory has two distinct kinds of activity:

- **Sections 3–4 (hands-on, “live”)**: you will interact directly with a *local, harmless stand-in* for the real bucket, using the same commands and the same protocol (S3 API) that a real investigator would use against the real cloud provider.
The goal here is not to find evidence but to build an intuition for *what kind of information the API itself can give you* (object listings, metadata, hashes), so that later, when reading the real exports, you understand exactly how that information was originally produced and what its natural limitations are.
- **Sections 5–6 (analysis, “the actual case”)**: you will examine the exports below, which represent what NovaDynamics' cloud provider actually handed over for this case. This is where the real investigation happens.

1.3 Evidence & Access Manifest

Artefact	Format	Description
cloudtrail_events.json	JSON	Management/data-plane event log export, provided by the CSP
s3_object_versions.json	JSON	Metadata export: versions, timestamps, region tags for the affected object
data_transfer_metrics.json	JSON	Independent billing/data-transfer-out report, hourly granularity
Live sandbox access	MinIO (Docker)	A local S3-compatible emulation of the bucket, for direct hands-on interaction

Table 1: Evidence and access manifest for Operation SILENT BUCKET.

• Warning

All JSON artefacts must be treated as evidence exports. Verify the integrity of each file before analysis by computing its SHA-256 hash and recording it in your chain-of-custody log.

No artefact may be modified in place; work only on copies.

Note that a hashing/query tool such as `jq` (Section 5) does not, on its own, make an analysis forensically sound. Keep this distinction in mind: it will resurface in Section 8.

2 Tool Procurement — A Role-Based Exercise

Before the analysis begins, each role must contribute to a documented tool selection decision. Three capability areas have been identified as immediate requirements for this case.

2.1 Required Capability Areas

Area 1: Cloud API Interaction

Tools must allow listing, retrieving, and hashing objects from a cloud object store via command line, without requiring direct infrastructure access.

Area 2: Structured Log Analysis

Tools must parse and query JSON-formatted event logs, filtering by event type, timestamp, and actor.

Area 3: Local Cloud Emulation

A capability to reproduce a minimal cloud object-storage API locally, for training purposes, without incurring cost or requiring a real provider account.

2.2 Background: Object Storage, S3, and Why It Matters Here

Before listing the tools, it is worth clarifying two things that later sections assume you already understand.

What “S3” actually is. S3 (Simple Storage Service) is the name of Amazon’s object storage product, but over time its HTTP API has become a de-facto *industry protocol*: “S3-compatible” storage is now offered by most cloud providers and by open-source software, regardless of who actually built it.

Unlike a filesystem, an object store has no directories in the traditional sense: every object is stored flat inside a *bucket*, addressed by a *key* (its name/path string) and accompanied by *metadata* (size, content type, last-modified time, and optionally custom key-value tags).

Whenever this laboratory says “interact with the bucket”, it means issuing HTTP requests against this API normally through the `aws cli`.

Why we interact with S3 directly at all. NovaDynamics’ actual cloud provider is, of course, not accessible to us — we only have the exports listed in Section 1.4. Section 4 of this laboratory has you interact with a *local stand-in* bucket instead. This is not redundant busywork: it is the only way, short of a real provider account, to see first-hand what the S3 API actually returns for an operation like “list objects” or “get object metadata”. That first-hand experience is what lets you later judge, in Sections 5–6, whether a given export is complete, plausible, or has gaps.

2.3 Background: What Docker Is, and Why We Use It Here

Docker lets you run a piece of software (here, MinIO) inside an isolated *container*, without installing it directly onto your VM's operating system and without it interfering with anything else on the system.

You do not need to understand Docker's internals for this laboratory; you only need three operations, all of which are shown step by step in Section 3: *run* a container (start the software), *list* running containers (check it is alive), and *stop/remove* a container (clean up afterwards).

We use Docker here purely as a convenient, disposable way to stand up a local S3-compatible server.

2.4 Pre-approved Tools

aws cli

Standard command-line interface for interacting with S3-compatible APIs; usable against both real endpoints and local emulators.

jq Command-line JSON processor, for querying and filtering the CloudTrail-style log export.

MinIO (Docker)

Local, open-source, cost-free emulation of an S3-compatible object store; run inside the same CAINE/Kali VM already used in previous laboratories.

sha256sum

Already used in Laboratories 2 and 3, reused here for verifying object integrity after acquisition.

3 Setting Up the Forensic Environment

This laboratory can reuse the CAINE14/Kali VM already configured in Laboratory 2.

For completeness, and so that this laboratory can be carried out on its own, even if you no longer have that VM available, the full environment setup is summarised again below.

3.1 Quick VM Setup via Pre-built .ova Image

If you do not already have a working CAINE14/Kali VM, the fastest path is to import an official pre-built .ova appliance rather than installing from an ISO. This is the recommended route if you are starting from scratch for this laboratory.

1. **Download.** Get the official Kali Linux VirtualBox .ova image from <https://www.kali.org/get-kali/#kali-virtual-machines> (choose the VirtualBox variant). Verify the download's checksum against the value published on that same page before proceeding **do not skip this step**; it is the same integrity-verification principle used throughout this course for evidence, applied here to your own tooling.
2. **Import.** Open VirtualBox → *File* → *Import Appliance* → select the downloaded .ova file → *Next*.
3. **Adjust resources before finishing the import.** On the settings review screen, increase **RAM to at least 4096 MB** (the default is often lower) — see the warning below for why this matters more here than in previous laboratories.
4. **Enable nested virtualisation.** After import, before first boot: select the VM → *Settings* → *System* → *Processor* tab → tick “**Enable Nested VT-x/AMD-V**”. This single setting is what allows Docker (Section 3.3) to run correctly inside a VM; without it, Docker containers may fail to start or run at very degraded performance.
5. **First boot and login.** Start the VM. Default credentials for the official Kali .ova are `kali / kali`. Change the password immediately after first login (`passwd`) if this VM will be kept beyond the laboratory session.
6. **Confirm networking.** Once logged in, verify the VM has a working network connection before continuing: run `ip a` (you should see an interface with an assigned IP address, not only `lo`) and then `ping -c 3 8.8.8.8` to confirm outbound connectivity.

- **Warning**

If your VirtualBox host does not expose “Enable Nested VT-x/AMD-V” at all (greyed out), your host CPU or hypervisor may not support

nested virtualisation. In that case, Docker inside this VM is not guaranteed to work reliably, and you should flag this to your team.

3.2 Base Forensic VM — Distribution Choice and Write-Blocking (recap from Laboratory 2)

If you are instead reusing an existing installation rather than the quick `.ova` route above, the following recap still applies:

Choice of distribution

CAINE 14 “Lightstream” *or* Kali Linux (Forensic Mode). Either is acceptable; both are pre-approved.

VM configuration

RAM: 4GB minimum (3GB absolute minimum — see the warning below regarding Docker’s additional memory footprint).

Network: host-only or NAT adapter, as long as outbound connectivity works (Docker needs to pull images from the internet the first time each is used).

Write-blocking

On CAINE14, devices are read-only (`RO=1`) by default via `rbfstab`; unlock explicitly with `blockdev -setrw` only when needed. On Kali, write-blocking is *not* automatic: run `sudo blockdev -setro /dev/sdX` on any evidence device before contact.

• Warning

This laboratory does not involve seized physical media, so write-blocking procedures are not exercised directly here — but the same VM, once configured, is reused for the Docker/MinIO setup below. 4GB of RAM should be treated as a firm minimum, not merely a suggestion: Section 3.3 adds a containerised service on top of the base VM’s own overhead, and low-RAM VMs have been observed to make the container unstable or very slow to respond.

3.3 Installing Docker in the Forensic VM

Run each of the following lines **one at a time**, waiting for each to complete before typing the next, **do not paste all four lines** as a single block: on

some terminal configurations (observed on Kali’s default `zsh`), pasting multi-line blocks can trigger bracketed-paste artefacts that corrupt the input and silently skip commands.

```
sudo apt update
```

```
sudo apt install docker.io
```

```
sudo systemctl enable --now docker
```

```
sudo usermod -aG docker $USER
```

- **Warning**

Stop here and restart your session. The `usermod` command above only takes effect the *next* time you log in — it does not apply retroactively to your current shell. Log out and log back in (or reboot the VM) before continuing, otherwise every subsequent `docker` command in this section will fail with a permission error unless prefixed with `sudo`.

After logging back in, confirm Docker actually works before moving on:

```
docker run hello-world
```

This should print a short “Hello from Docker!” confirmation message. If it hangs, times out, or errors out, revisit the nested-virtualisation setting from Section 3.1 before proceeding further. This is the most common cause of failure at this exact step.

3.4 Setting Up a Local S3-Compatible Emulator (MinIO)

With Docker confirmed working, start MinIO, that is a local, open-source server that speaks the same S3 API as a real cloud provider, so that the `aws cli` commands you will run in Section 4 work identically to how they would against a real bucket.

```
docker run -d --name minio \  
  -p 9000:9000 -p 9001:9001 \  
  -e "MINIO_ROOT_USER=minioadmin" \  
  -e "MINIO_ROOT_PASSWORD=minioadmin" \  
  minio/minio server /data --console-address ":9001"
```

Verify the container started correctly and *stayed running* (this matters: some containers start and then immediately exit due to a configuration error — a running container is not the same as a successfully created one):

```
docker ps
```

You should see a line for `minio` with status `Up`.

If instead `docker ps` shows nothing, run `docker ps -a` (which also lists stopped/exited containers) and then inspect why it exited:

```
docker logs minio --tail 20
```

The log output should show the MinIO startup banner (version, API/console addresses) with no license or authentication error.

- **Warning**

MinIO is a *local, open-source S3-compatible emulator*, not a real cloud environment. It is used here only to give you first-hand experience with the interaction pattern (CLI → API → object store), not to reproduce the full complexity of a real provider. All log-based analysis in this laboratory relies on artefacts provided separately (Section 5), reflecting the fact that a real forensic investigator receives log exports, not raw infrastructure access.

3.5 Configuring `aws cli` Against the Local Endpoint

The `aws cli` is a generic client: the same tool talks to a real AWS account or to any S3-compatible endpoint, depending only on which credentials and endpoint URL you give it. Install it first if not already present (it ships by default on CAINE14; on Kali it must be installed explicitly):

```
sudo apt install awscli
```

Then configure it with the MinIO root credentials. Run each line separately, exactly as in Section 3.3 — do *not* paste them as one block:

```
aws configure set aws_access_key_id minioadmin
```

```
aws configure set aws_secret_access_key minioadmin
```

```
aws configure set region eu-west-1
```

Verify the credentials were actually stored before moving on — do not assume the three commands above succeeded silently:

```
aws configure list
```

You should see non-empty (masked) values for `access_key` and `secret_key`. If either shows as empty, repeat the corresponding `aws configure set` command above.

Finally, define the alias used throughout the rest of this laboratory:

```
alias awslocal="aws --endpoint-url=http://localhost:9000"
```

• Warning

This `alias` only lasts for the current shell session. If you close the terminal or open a new one, you will need to redefine it (or add it to `~/.bashrc` / `~/.zshrc` if you want it to persist).

Question

Why must every command in this section explicitly specify `-endpoint-url`?

What would happen if a student forgot this flag, and why is this exact mistake also a realistic risk during a genuine cloud investigation involving multiple provider accounts or regions?

Your answer:

4 First Contact — Direct Interaction with the Bucket

The purpose of this section is *not* to find evidence but the purpose is to build a first-hand, concrete understanding of what the S3 API actually gives you when you ask it for something, so that in Sections 5–6, when you read exports that someone else produced, you know exactly what process generated them and what it could or could not have captured.

4.1 Creating the Test Bucket and Uploading Objects

Run each command separately and check its output before moving to the next one:

```
awslocal s3 mb s3://novadynamics-billing-archive
```

This creates an empty bucket. “mb” stands for “make bucket”; a bucket is the top-level container for objects, roughly analogous to a top-level folder, except that S3 buckets cannot be nested inside one another.

```
echo "test content" > contract_2026_0091.pdf
```

(You are creating a harmless placeholder file locally, standing in for the real contract document. Its actual content does not matter for this exercise.)

```
awslocal s3 cp contract_2026_0091.pdf s3://novadynamics-billing-  
archive/
```

This uploads (“puts”) the object into the bucket, under the key `contract_2026_0091.pdf`.

```
awslocal s3 ls s3://novadynamics-billing-archive/
```

This lists the bucket’s contents — you should see your uploaded file, its size, and its upload timestamp.

4.2 Integrity Verification of the Retrieved Object

```
awslocal s3 cp s3://novadynamics-billing-archive/contract_2026_0091  
.pdf ./retrieved.pdf
```

```
sha256sum retrieved.pdf
```

Question

You have just computed a hash of an object retrieved through the cloud provider’s own API — you did not access any underlying disk. What exactly does this hash prove, and what does it *not* prove, compared to a hash computed on a bit-for-bit disk image as in Laboratory 2?

Your answer:

4.3 Listing Object Metadata

```
awslocal s3api head-object --bucket novadynamics-billing-archive --  
key contract_2026_0091.pdf
```

This is the same category of operation — an object metadata lookup — that produced the `s3_object_versions.json` export you will analyse in Section 6.

Take a moment to compare the fields you see here with the fields inside that export file; they should look structurally similar, because they come from the same kind of underlying API call.

Question

Compare the metadata fields available here to the metadata ExifTool extracted in Laboratory 2 (Section 7.1). What forensically useful information is missing, and why?

Your answer:

5 Analysis of Provided Log Material

This is where the actual investigation begins. Everything from this point onward uses only the exports listed in Section 1.4 — not the sandbox from Sections 3–4, which has already served its purpose.

• Warning

A quick note on tool soundness before you start querying: `jq`, used as below, only reads the input file and prints to your terminal, it never modifies the file in place.

This makes it compatible with forensic soundness *if you follow the right procedure*: hash each JSON file before you touch it (as required in Section 1.4), never redirect `jq`'s output back onto the original file, and work from a copy. Note, though, that this soundness comes from *how you use the tool*, not from `jq` being a certified forensic tool in the way Autopsy or Volatility are; it is a generic, general-purpose JSON processor. Keep this distinction in mind for Section 8.

5.1 Initial Survey

```
jq '.Records[] | {eventTime, eventName, sourceIPAddress,
  userIdentity: .userIdentity.userName}' cloudtrail_events.json
```

This query extracts, from every event record, only the four fields most relevant for a first pass: when it happened, what happened, from where, and by whom. Scan the output chronologically.

Question

List, in order, every event associated with the `svc-billing-sync` role on 19 May 2026. What is anomalous about the `sourceIPAddress` of the first event in that sequence, and how would you have discovered this anomaly systematically rather than by simply reading the note field provided here?

Your answer:

5.2 The Missing Data-Plane Events

Recall from lecture the distinction between *management-plane* events (actions on the cloud resources themselves: creating a bucket, changing its policy, listing its contents) and *data-plane* events (actions on the actual data inside those resources: reading or writing individual objects).

Whether data-plane events are logged at all is a separate, opt-in configuration choice on most cloud platforms, it does not come "for free" alongside management-plane logging.

Question

Between the two `PutBucketPolicy` events at 03:16:22Z and 03:45:51Z, no `GetObject` events appear anywhere in the log. Explain precisely *why* this is possible in a real cloud environment, referencing the distinction between management-plane and data-plane events. Is the absence of these events itself evidence, and if so, of what?

Your answer:

5.3 Cross-Validating the Finding with a Second, Independent Method

Recall from lecture that, during acquisition, using more than one hashing algorithm at once strengthens confidence in an integrity claim: if two independent computations agree, an error or manipulation in just one of them becomes far less plausible.

The same logic applies to analysis, not just acquisition — a finding that depends entirely on a single tool's correct behaviour is weaker than one confirmed by two independent tools that parse the data in different ways.

Confirm the absence of `GetObject` events from Section 5.2 using a tool that does not depend on `jq`'s JSON parser at all:

```
grep -c '"eventName": "GetObject"' cloudtrail_events.json
```


This performs a dumb, literal text search for the string, entirely independently of `jq`'s JSON interpretation. The result should be 0, agreeing with what you observed in Section 5.2.

Question

The `grep` command above and the `jq` query in Section 5.1 use fundamentally different methods to read the same file (naive text search vs. structured JSON parsing). Why does their agreement here strengthen your finding more than simply re-running the exact same `jq` query a second time would? Can you construct a (contrived) scenario in which `jq` and `grep` would legitimately disagree on this kind of question, even with perfectly valid, unaltered JSON?

Your answer:

Since the log export itself has a blind spot exactly where we need it most, we turn to a second, independent artefact: a billing/data-transfer report. Billing systems and audit-logging systems are usually built and operated independently of one another inside a cloud provider, precisely because they serve different purposes — which is exactly what makes cross-referencing them useful here.

```
jq '.hourly_data_transfer_out_MB[]' data_transfer_metrics.json
```

Question

Using `data_transfer_metrics.json`, identify the anomalous hour and estimate — as precisely as the data allows — the order of magnitude of data transferred out of the bucket during the un-logged window. This source was **not** disabled by the same bucket-policy manipulation that suppressed `GetObject` logging — why not? What does this tell you about designing a logging architecture that is resilient to a single point of compromise?

Your answer:

5.4 Reconstructing the Attacker's Actions

Question

Write a short timeline (bullet list, with UTC timestamps) reconstructing the attacker's actions from first anomalous login to policy restoration, explicitly marking which steps are supported by direct log evidence and which are inferred from indirect/circumstantial evidence (Section 5.4). Why is this distinction important if this timeline is later presented in court?

Your answer:

6 Redundancy and Primary Copy Identification

Cloud storage is, by default, redundant: providers routinely replicate objects across multiple physical locations (and often across regions) for durability and disaster-recovery purposes, with no action required from the customer. From a forensic standpoint this redundancy is a double-edged sword: it means the data is unlikely to be permanently lost, but it also means that “the same file” may legitimately exist in more than one place at once, at slightly different

points in a replication cycle — and only one of those copies was actually the one interacted with at the time of the incident.

6.1 Inspecting the Version Export

```
jq '.versions[] | {versionId, region, lastModified, etag, replicationStatus}' s3_object_versions.json
```

Question

Three entries are listed for `contract_2026_0091.pdf`. Which one is the primary, authoritative copy at the time of the incident, and which is a replica? Justify your answer using at least two independent fields from the export — do not rely on `lastModified` alone.

Your answer:

6.2 The Replication Trap

Question

A colleague on your team argues that the `us-east-1` entry must be the “real” exfiltrated version, since it has the most recent timestamp. Explain why this reasoning is flawed, and what evidence in the export directly refutes it.

Your answer:

6.3 Chain of Custody Documentation

Question

Draft the chain-of-custody entry (in the same format used in Laboratory 2, Section 6.2) for the primary copy identified above. What field(s), specific to a cloud/versioned-object context, do you need to record here that had no equivalent in the Laboratory 2 disk-image chain of custody?

Your answer:

7 Cloud Service Model Comparison

Question

NovaDynamics' billing service actually spans two components: the automated sync job (running on a dedicated IaaS virtual machine) and a customer-facing web portal (deployed on a managed PaaS platform). For *each* of the two components, answer:

- (a) What categories of forensic evidence would realistically be obtainable directly, without CSP cooperation?

- (b) What would require formal cooperation from the cloud service provider?
- (c) If this same billing service had instead been built entirely on a third-party SaaS platform, what would change in your answers to (a) and (b)?

Your answer:

8 Legal & Trust Module

Question (a)

A replica of the exfiltrated object resides physically in `us-east-1` (United States), while NovaDynamics and its customers are based in the EU.

To which broader problem of operating in the cloud does this situation correspond?

Your answer:

Question (b)

All of the evidence in Sections 5–6 was provided to you as an export generated by the cloud provider’s own tooling. You have no way to physically verify that this export was not altered before being handed to you.

- (a) Why can this reliance be challenged in a court of law, even if the cloud provider acted in good faith?
- (b) Based on principles already covered in this course (e.g. hashing, chain of custody, repeatability), what would need to be true about how this export was produced for you to defend it as being just as trustworthy as a bit-for-bit disk image acquired directly, as in Laboratory 2? Given what you do *not* control in a cloud environment, is that standard actually achievable here?

Your answer:**Question (c)**

Throughout Section 5 you used `jq`, a general-purpose JSON tool with no forensic certification, to analyse evidence exports — and in Section 5.3 you cross-validated one finding using `grep` as an independent second method. Reconcile the following: what, precisely, makes an analysis “forensically sound” — the tool used, or something else? What did the cross-validation step actually add that simply re-running the same `jq` query again would not have?

Your answer:

9 Wrap-Up

Question

In half a page, summarize the peculiarities of this investigation: what evidence categories were unavailable to you here that would have been trivial to obtain in a disk- or memory-based investigation, and what new categories (if any) did the cloud environment make available that a traditional investigation could not have provided?

Your answer: